

Which AI should you **actually** use?

Choosing the product, the mode and the model — in that order.

What this **covers**

- 01 Why “which model is best?” is the wrong question
- 02 The four layers — task, capability, product, model
- 03 Seven questions that decide it
- 04 Three practical rungs and why price gaps vary
- 05 What people get wrong, and how to test properly

The strongest model is often the **wrong** one

Product

Where the work happens

A chat window, a research mode, a coding agent, an automated pipeline.

Model

How capable the engine is

Fast, balanced or frontier. The part everyone argues about.

Tool

What the system can actually do

Search the web, open your files, run code, edit a repository.

A weaker model inside the right product beats a stronger model in the wrong one.

Four layers. Pick the model **last**

01

Task

What is the output, and who reads it?

02

Capability

Search, long documents, vision, code execution.

03

Product & tools

Chat, research mode, coding agent, pipeline.

04

Model tier

Only now. And rarely the frontier.

Each layer removes candidates. By the time you reach the model, most of the decision is already made.

THE METHOD

Seven questions

Stop at the first one that decides it.

First, what the task **requires**

01	Does the answer depend on information that can change?	Yes → live search is mandatory, and citations if anyone checks the claims. A model without search answers confidently and wrongly.
02	Is the work tied to a particular environment?	Drive, SharePoint, a repository, a database. Whatever reaches the data wins — integration outweighs a modest quality difference.
03	How many files, and how long?	A few: attach them. One very long one: long-document handling. Dozens: file search. Thousands: an indexing project, not a chat.
04	What is in the input besides text?	Charts, scans, audio, video. Check this before comparing intelligence — a smarter model that cannot see the chart is useless.

Then, what you are **optimising for**

- | | | |
|----|--|--|
| 05 | How hard is the thinking, honestly? | Routine work on the flagship wastes money. Client-facing work on the cheap tier costs more the first time someone has to fix it. |
| 06 | Which constraint actually binds – quality, speed or cost? | One usually dominates. Name it, optimise for it, and say plainly what was traded away. |
| 07 | Is the data sensitive or regulated? | A hard gate, not a preference. Use what the organisation has approved, even when it is the weaker tool. |

Three practical **rungs**, without fixed price bands

RUNG	WHAT IT IS FOR	COST TENDENCY
Fast / light	High volume and repetitive: classification, extraction, short summaries, quick answers.	LOWEST
Balanced	Most real work: drafting, analysis, everyday coding, document work. The working default.	HIGHER
Frontier	Hard reasoning, architecture, long autonomous runs, anything expensive to get wrong.	HIGHEST

Provider price gaps vary too much for one multiplier. Verify current pricing, then measure cost per successful task. In consumer apps, use the thinking control: fast, think harder, maximum.

What each provider calls them **today**

RUNG	OPENAI	ANTHROPIC	GOOGLE
Fast	GPT-5.6 Luna	Claude Haiku 4.5	Gemini 3.5 Flash-Lite
Balanced	GPT-5.6 Terra	Claude Sonnet 5	Gemini 3.8 Flash
Frontier	GPT-5.6 Sol · GPT-6 Astra	Claude Opus 5 · Fable 5.1	Gemini 3.1 Pro · Deep Think

This table is the part of the deck that goes stale. It was accurate on 2026-09-22 and should be checked against each provider's own documentation before it is shown again.

Six rules of **thumb**

Cost per successful task, not per token

A cheap model that retries twice and needs a human fix is the expensive one.

A big context window is not comprehension

Fitting the text in is not the same as reasoning well over all of it.

Benchmarks shortlist, they do not decide

Run three real examples through two candidates. Ten minutes beats a leaderboard.

“Analyse this PDF” is not one task

A summary, one exact fact, a table and a judgement route to four different setups.

“Write code” is four different jobs

One function, a bug hunt, a repo-wide refactor and an architecture review differ.

Do not switch provider for a few points

Connectors, ecosystem fit and the subscription already paid for matter more.

The test that beats every **leaderboard**

01

Take three real examples

Not a toy prompt. Three tasks from actual work, with the same files and constraints.

02

Run them through two candidates

Shortlisted from the decision tree — not from a ranking table.

03

Judge blind, then pin the winner

Fix the setup, fix the output format, and re-test when something changes.

Ten minutes of this reflects your own house style, your own data and your own quality bar. No published benchmark does.

The strongest model is **rarely** the right one.

Route by what the task requires, not by what scores highest.

NEXT STEP

Install the skill. Ask it **your** question.

The which-ai-model skill runs this decision tree for you and verifies current model names by live search — so the answer is never read from a stale table.

github.com/houseofichigo/skill-resources/tree/main/skills/which-ai-model